SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 05

Agentic AI Fundamentals – Part 1

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@sliit.lk | Course web: courseweb.sliit.lk

Today's Roadmap – 3 Hours

Hour 1	**Frameworks & Fundamentals** Introduction to Agentic AI | Frameworks | Fundamental knowledge about Models
Hour 2	**MODEL + TOOLS + LOOP + GOALS** Effective tool usage in Agentic AI
Hour 3	**Composition** Quick intro to the LABS

Short breaks after each hour • Questions welcome at any point

PART 1 OF 4

Fundamentals Of LLMs

- How LLMs work
- Limitations of LLM workflows
- What is agentic AI
- Agentic AI spectrum

Agent vs Model

Prompt:
Find me a flight to Singapore next Friday under $400 and hold a seat.

Model:
Here are some tips for finding cheap flights...

Agent:
Held seat 34C on TR 561, Confirmation #8842

Small pieces of words combine to form sentences

Tokens

Tokens are the foundational units of text processing in NLP, created by breaking down raw text into smaller components like words, subwords, or even characters.

Embeddings

Embeddings are the result of training vectorized tokens to capture semantic relationships between words. Unlike raw vectors, embeddings position similar words (e.g., "king" and "queen") closer in vector space, allowing models to understand context and meaning.

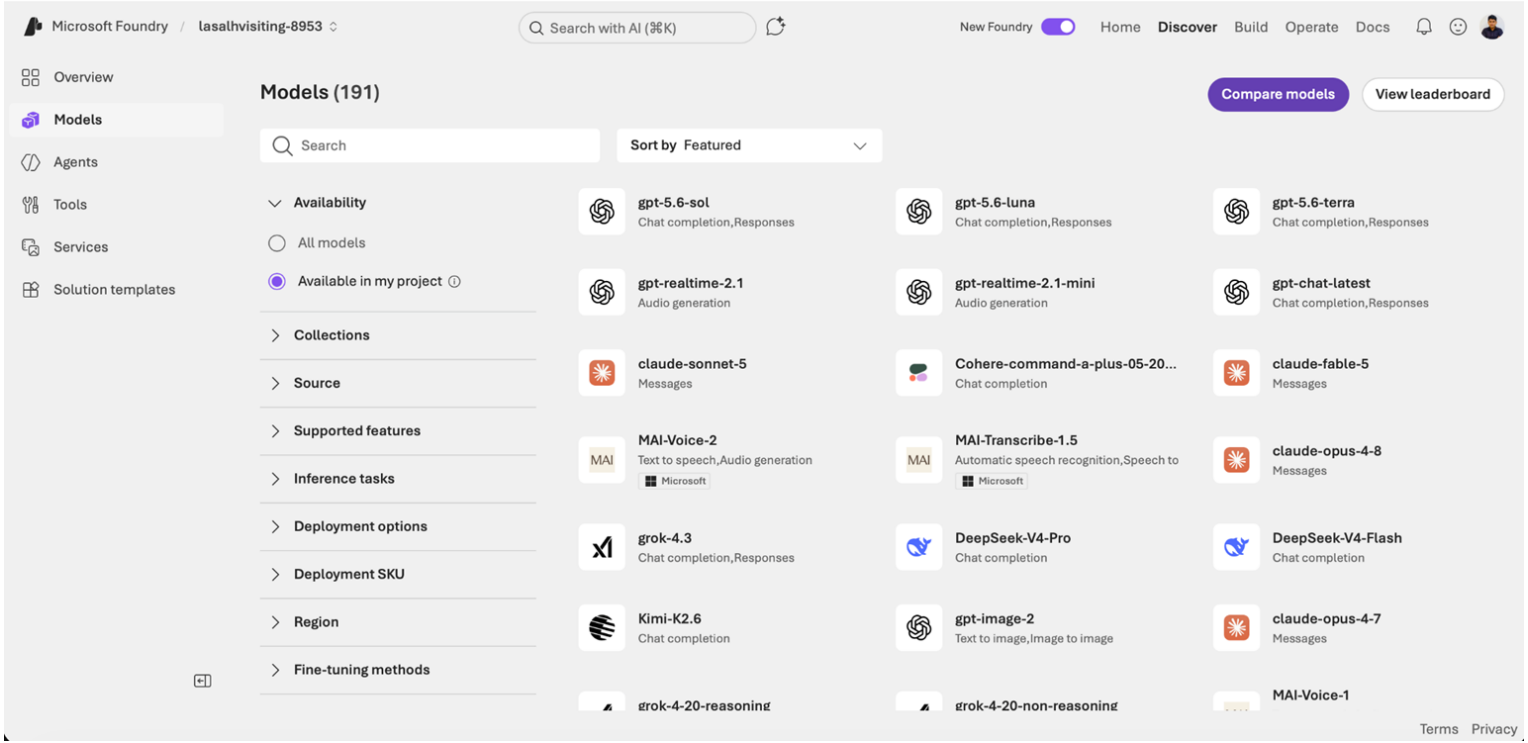
https://medium.com/@saschametzger/what-are-tokens-vectors-and-embeddings-how-do-you-create-them-e2a3e698e037

What is an LLM, really?

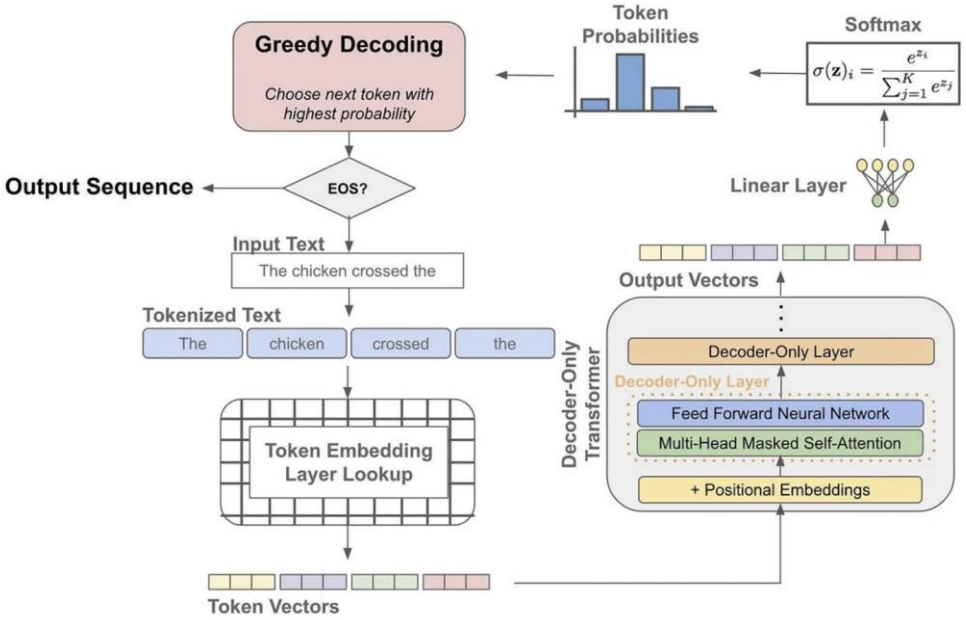
Definition: A neural network trained to predict the next token given the previous tokens. Usually the first set of seed text (user prompt) and the system message is provided.

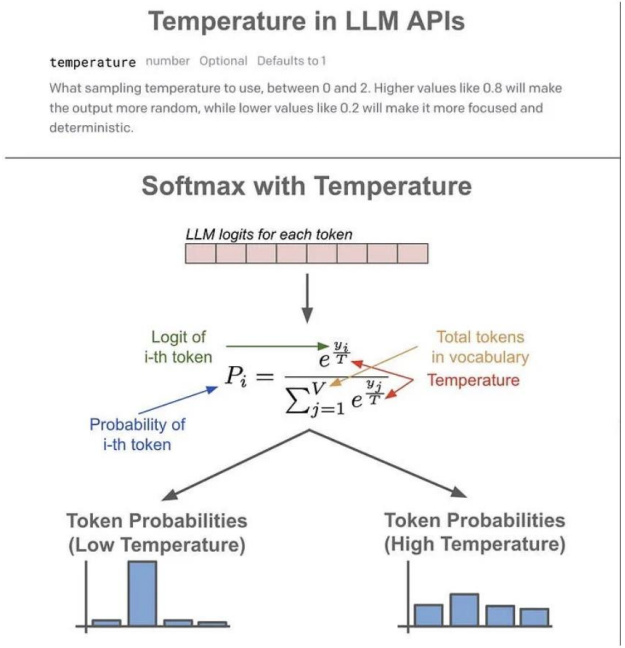
- "Token" ≈ a chunk of text (~¾ of a word). Text is split into tokens first.
- Trained in two broad stages (high level):
  - **Pre-training** — read enormous amounts of text → learn language & world patterns.
  - **Alignment / instruction-tuning** — learn to *follow instructions* and be helpful.
- At inference it is, fundamentally, a **function**: `tokens in → next token out`, applied repeatedly to generate text.

Model Catalog



What is an LLM, really?





From hidden state to a probability

At each step the network emits a vector of logits — one raw score zi for every token i in the vocabulary V.

- Logits become a probability distribution via the **softmax**:

$$p_i = \frac{e^{z_i}}{\sum_{j \in V} e^{z_j}}$$

- Each $p_i \in (0, 1)$ and $\sum_i p_i = 1$ — a valid distribution.
- Exponentiation makes bigger logits dominate; it's monotonic, so ranking is preserved.

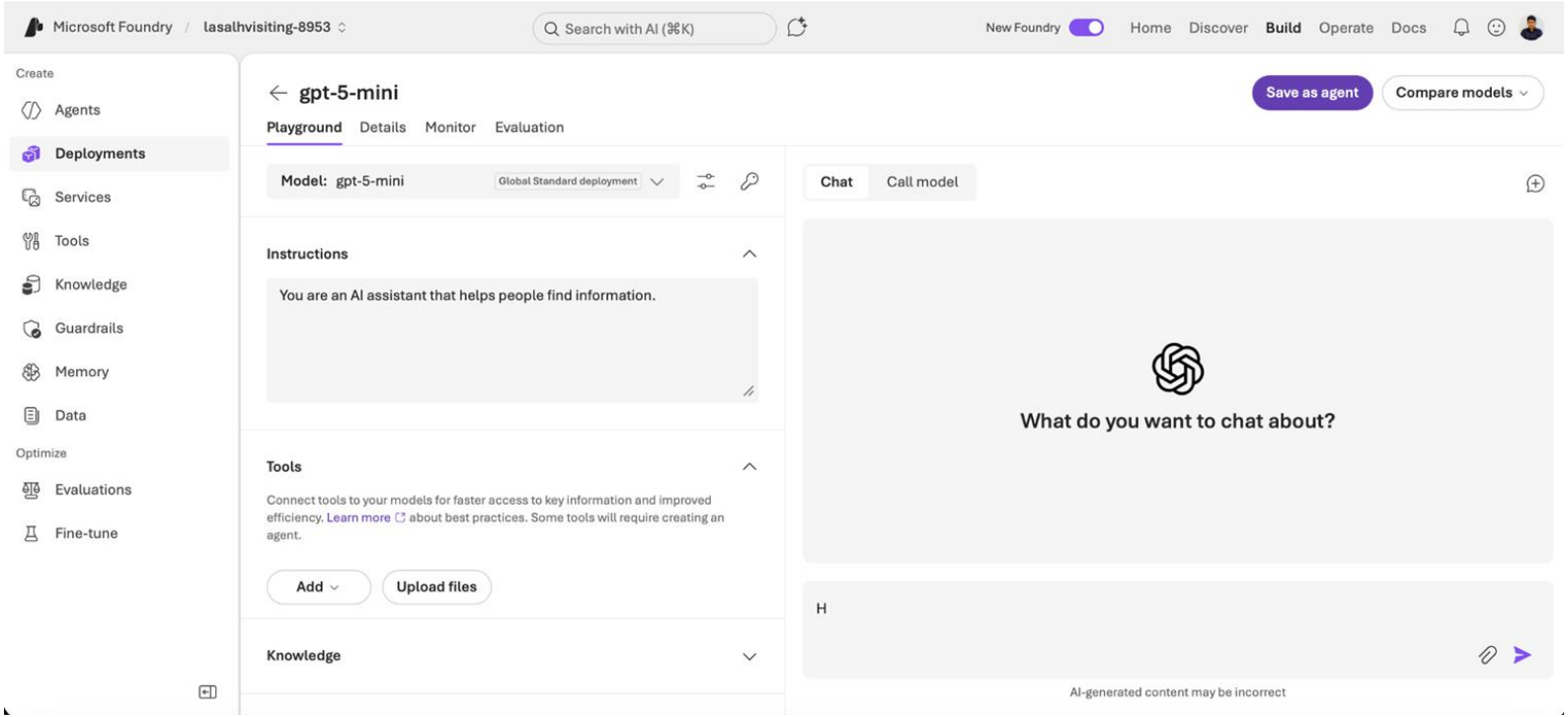
Temperature sharpening vs flattening

- Temperature T rescales the logits **before** softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T \to 0$: the largest logit dominates → distribution collapses to **argmax** (greedy, deterministic).
- $T = 1$: the model's "natural" distribution.
- $T \to \infty$: logits flatten → **uniform** (maximum randomness).

Let's go for a quick playthrough



The three defining limitations

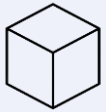
- There are 3 defining limitations when it comes to using models alone
- Almost everything we build this term compensates for these:

Limitation	What happens there	
Stateless	No memory between calls	If you want the model to "remember," you must store and replay it.
Ungrounded	No live data; only training + your input	To answer about your documents or current facts, you must bring the data to the model (retrieval) or give it tools to fetch it.
Probabilistic	Improvises; "best" not guaranteed	We control this with the **temperature**

Evolution from standalone models to Agentic AI

AI that can

Create for you

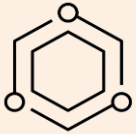


Models

- Text summarization
- Text synthesis
- Pattern matching

AI that can

Retrieve for you



Assistants

- Information retrieval
- Prescriptive tasks
- Single step or rule based processes

AI that can

Do for you

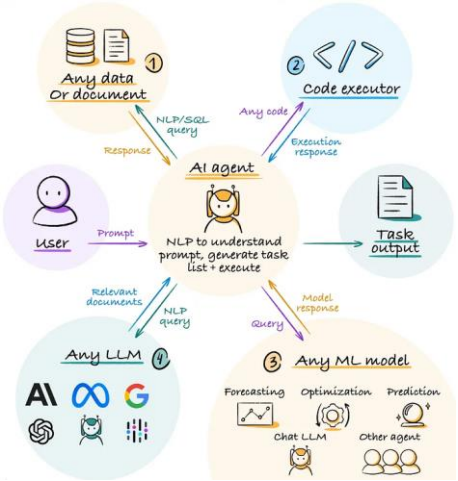
Agents

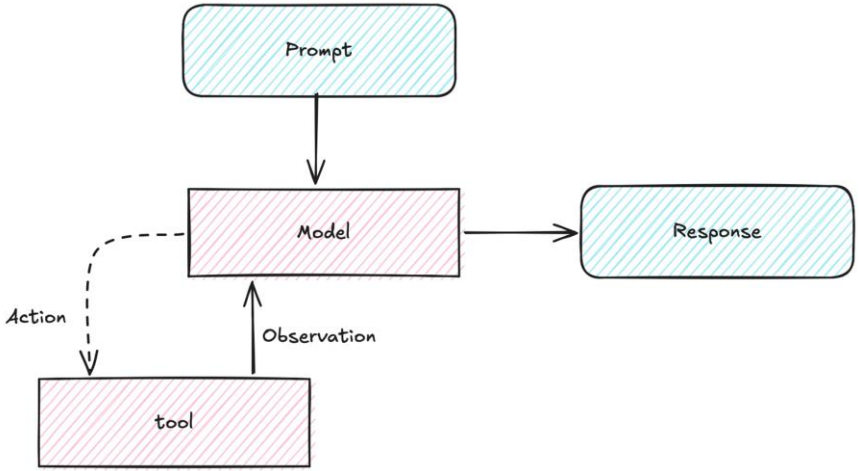
- Multi step processes
- Autonomous actions
- Self-corrections

- The evolution from probabilistic to agentic and how we overcome the limitations of LLMs

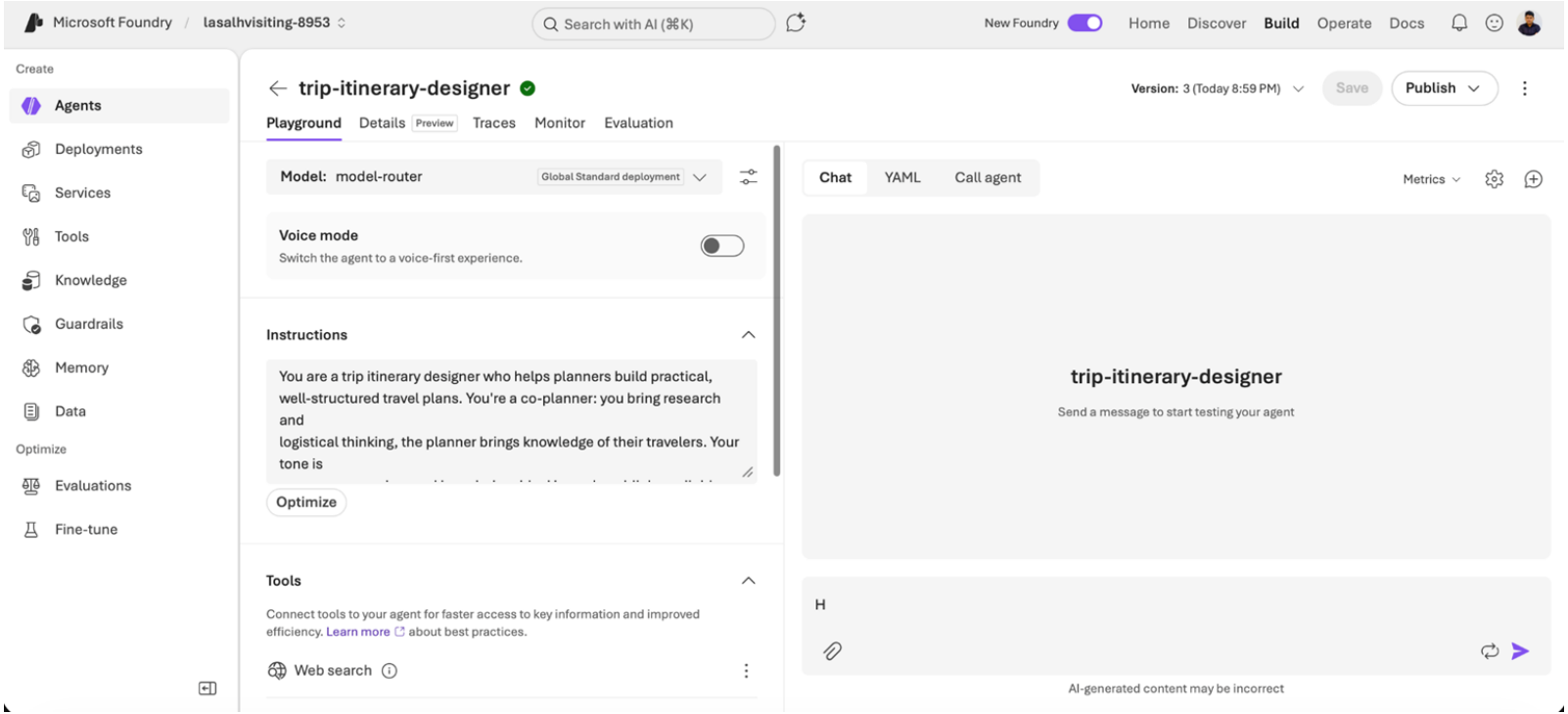
What is an Agent?

Definition: An LLM directing a loop: it observes state, decides on actions (tool calls), a runtime executes them, and results feedback.





Quick demo on playground



The Agency Spectrum

Definition: "Agent" is not binary. A useful ladder — each rung hands the LLM more control over control flow:

Rung	LLM decides…	Example
LLM call	Content of one output	summarize this email
Workflow / chain	fixed code path with LLM steps	translate → summarize → format
Router	which of N fixed paths	triage: billing vs tech support
Autonomous agent	Which tools, what order, when to stop	Book me a flight to London tomorrow

Do You Actually Need an Agent?

- Agents buy flexibility and pay with latency, cost, and non-determinism
- Every LLM decision is a chance to be wrong

Ask yourself…	Then reach for…
Can you write the steps down in advance?	workflow, not agent.
Is the output space small and known?	router/classifier.
Do you genuinely need dynamic multi-step decisions against an unpredictable environment?	agent with the least autonomy that works.

Checkpoint 1 – Justify which pattern you would use

1. You are building an application that takes a daily news article in Spanish, translates it into English, summarizes it into three bullet points, and finally formats the output into a specific JSON structure for your database.

2. A user prompts your application with: "Find me a flight to London tomorrow, book a 4-star hotel near the city center, and check if I need an umbrella based on the forecast." The system must interact with external booking APIs and weather services to fulfill the request.

3. Your company receives thousands of customer emails daily. You need a system that reads each incoming email and simply categorizes it into one of three buckets: "Billing Issue," "Technical Support," or "General Inquiry."

Pair up — 3 minutes — then we discuss answers together.

Checkpoint 1 – Answers

1. Workflow / Chain

You can write the steps down in advance (Translate → Summarize → Format). It relies on a fixed code path with specific LLM steps rather than autonomous decision-making.

2. Autonomous Agent

This requires dynamic multi-step decisions against an unpredictable environment (flight/hotel availability). The LLM must observe state, decide which tools to use and in what order, and execute them.

3. Router / Classifier

The output space is small and known (only three categories). The LLM just needs to decide which of the N fixed paths the input belongs to.

Rule of thumb — choose the least autonomy that still solves the problem.

PART 2 OF 4

ReAct & Tool Calling

- ReAct tool calling
- Tool execution lifecycle

ReAct: why interleaving beats planning ahead

PROMPT: "Who is the current CEO of the company that owns LinkedIn, and what year did they become CEO?":

- **Plan-then-execute**: write the full plan, run it. Fragile — the plan can't react to what search actually returns (what if the owner changed? what if search returns a disambiguation page?).
- **ReAct (Yao et al., 2022 — Reasoning + Acting):** alternate Thought → Action → Observation, deciding each action after seeing the last observation:

```
Thought: I need the owner of LinkedIn first.          | reasoning (tokens)
Action:  search("who owns LinkedIn")                  | tool call
Observation: "Microsoft acquired LinkedIn in 2016"    | runtime result
Thought: Now I need Microsoft's current CEO…          | reasoning
Action:  search("Microsoft current CEO start year")   | tool call
Observation: "Satya Nadella, CEO since 2014"          | result
Thought: I can answer now.
Answer:  Satya Nadella, CEO since 2014.
```

The Messages

Definition: The LLMs has both the text completion and the reasoning capability and some of them offer tool calling capabilities also meaning it will emit both tokens as well as actions.

Message types

- System message - Tells the model how to behave and provide context for interactions
- Human message - Represents user input and interactions with the model
- AI message - Responses generated by the model, including text content, tool calls, and metadata
- Tool message - Represents the outputs of tool calls

https://docs.langchain.com/oss/python/langchain/messages
https://developers.openai.com/api/docs/guides/function-calling

The Bridge problem

LLMs emit tokens. Action

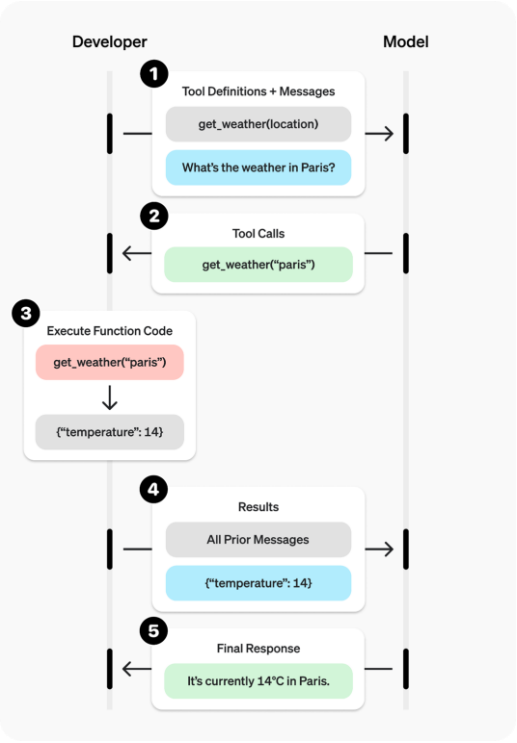
LLM emits both tokens as well as actions (DB calls, API calls, other function executions etc)

Naive approach

(Regex parsing JSON) is 80% reliable

Solution: Native Tool Calling (post-trained to emit structured calls).

Frame by Frame: Tool calling cycle



- Tools extend what agents can do letting them fetch real time data, execute code, query external databases, and take actions in the world.
- Under the hood, tools are callable functions with well defined inputs and outputs that get passed to a chat model. The model decides when to invoke a tool based on the conversation context, and what input arguments to provide.

```python
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the customer database for records matching the query.

    Args:
        query: Search terms to look for
        limit: Maximum number of results to return
    """
    return f"Found {limit} results for '{query}'"
```

Checkpoint 2 – Quick Activity

1. The Scenario: Brainstorm one specific real-world task where an AI agent cannot rely on its internal knowledge and must interact with an external system (e.g., checking live sports scores, sending an email, booking a meeting room).

2. Design the Signature: Give your tool a clear function name and list the exact input parameters (arguments) the LLM would need to extract from a user's prompt (e.g., book_room(room_id, start_time, duration)).

3. Determine the Return: What specific data type or confirmation must this tool return back to the LLM so it can formulate a helpful final response to the user?

Pair up — 3 minutes — then we discuss answers together.

PART 3 OF 4

Agentic Frameworks

- Agentic frameworks
- How to choose the correct framework

Agentic frameworks

Definition: An agentic framework is a software architecture designed to build, manage, and orchestrate autonomous AI agents.

Overcoming the "Stateless" Limitation

A raw LLM cannot pause a task, wait for an external system to process a job, and come back to it later. Agentic frameworks manage the "state" of a task. If a process takes hours or requires waiting for a human to approve an action, the framework parks the state and resumes it seamlessly.

Deterministic Control in a Probabilistic System

LLMs are probabilistic. Frameworks enforce structure. They ensure that if an LLM decides to delete a file, the framework intercepts that decision, validates the parameters, and can enforce a "human-in-the-loop" approval process before the action actually happens.

Cyclical Execution and Error Correction

The agent can execute a tool (like running code), read the error message, realize its mistake, and try a different approach without the user having to re-prompt it.

Multi-Agent Orchestration

Complex problems often require different skill sets. Instead of using one massive prompt to do everything, frameworks allow developers to build multi-agent systems.

Key components of an agentic framework

Models

The brain of the system. While the LLM is the most famous part, an agentic framework often uses a mix of models.

Tools

Tools give the LLM "hands" to interact with the outside world. The framework translates the LLM's text output into functional calls.

Memory

Memory provides the context necessary for the agent to maintain continuity across tasks and sessions.

Monitoring

This is the monitoring backend for the agents execution.

Prompts

The instructions and the response objects that govern the HUMAN, SYSTEM, TOOL, AI lifecycle

Middleware / Orchestration

These include the structured outputs, guardrails, HIL, Orchestration etc.

Popular agentic frameworks

Framework	Best at	Reach for it when…
LangChain + LangGraph *(our stack)*	biggest ecosystem of integrations; LangGraph: explicit, durable, resumable control flow	you want maximum transferability and production-grade control
Microsoft Agent Framework (succeeds AutoGen + Semantic Kernel)	multi-agent patterns with first class citizenship	deep Azure ecosystem alignment
LlamaIndex	data first: ingestion, indexing, retrieval quality knobs	RAG is the product and agents are a feature (inverse of LangChain's emphasis)
Google ADK	thorough multi agent orchestration with sequential pipelines, parallel workflows and feedback loops	quick prototyping and deep orchestration flows and handoffs
DSPy	*optimizing* prompts/pipelines programmatically (adjacent to agents)	you have evals and want the machine to tune the prompts

Popular agentic frameworks

Best Practices

- Start with Small-Scale Prototypes
- Incorporate Human Oversight
- Optimize Memory Usage
- Ensure Robust Monitoring and Observability
- Ensure security is not an afterthought

https://blog.jetbrains.com/pycharm/2026/06/top-agentic-frameworks-for-building-applications-2026/#best-agentic-frameworks-for-your-projects
https://www.exabeam.com/explainers/agentic-ai/agentic-ai-frameworks-key-components-top-8-options/

Next Week & How to Prepare

Lecture 06 — Knowledge & memory of AI agents: RAG, embeddings, retrieval, memory & the idea of tools

- Go through the LABs
  - Go through background material on the above topics
  - Play around on Azure AI Foundry